Lesson 4: Curves

Lesson 4: Curves 1
4.1 Parametric Curves 2
4.2 Ellipses 2
4.2.1 Properties of Ellipses 2
4.2.2 Midpoint Ellipse Algorithm 2
4.3 Polygons 3
4.3.1 Surface normal 3
4.3.2 Polygon soup 3
4.3.3 Meshes 3
4.4 Scan conversion 4
4.4.1 Scan-converting a line 4
4.4.2 Scan-converting a polygon 4
4.4.3 Sweep-line algorithm 4
4.4.4 Hidden surface removal 4
4.5 Rendering Curves in OpenGL 5
4.6 Two dimensional evaluators 8
Lesson 4 Discussion Session 10

4.1 Parametric Curves

This is a graph of parametric equation**.**
Parametric Equations defines a curve as a system of equations with an independent variable (t)
x = f(t)
y = g(t)
z = h(t)

General Parametric Framework

```
for t:0 -> 1
  x = f(t)
  y = g(t)
  z = h(t)
  add (x, y, z)
```

Bezier Curves and Surfaces

It was developed by French engineer Pierre Bezier for Renault automobile bodies.
It has a number of properties and easy to implement so it is widely available in various CAD and graphics package.

Properties of Bezier Curves

- ✓ It always passes through first control point i.e $p(0)=p_0$
- ✓ It always passes through last control point i.e $p(1)=p_n$
- ✓ Bezier Curves always lies within the convex hull of the control points
- ✓ Bezier blending function is always positive
- ✓ Bezier blending function is always 1

4.2 Ellipses

Defined as the set of points such that the sum of the distances from two fixed position(foci) is same from all points.

Ellipse is a plane curve surrounding two focal points, such that for all points on a curve, the sum of the two points is constant

4.2.1 Properties of Ellipses

- ✓ If we labeled distance from foci to any point on ellipse boundary as d_1 and d_2 the general equation of an ellipse can be written as follow

$$d_1+d_2=Constant$$

- ✓ Has major axis which is a straight line segment passing through both foci and extends up to boundary on both sides.
- ✓ Has minor axis that spans the shortest dimension of ellipse, it bisects the major axis at right angle in two equal halves

4.2.2 Midpoint Ellipse Algorithm

Algorithm used to draw ellipse in computer graphics.

Modified from Bresenham's algorithm

Advantages

- ✓ Only addition operations are required in the program loops, hence simple and fast implementation in all processors.
- ✓ Simultaneous calculations of pixel positions with parallel processors.

4.3 Polygons

A polygon is the space bounded by a set of edges between pairs of vertices in an ordered set.
They are structured, polygons maintain a hierarchy of structure
OpenGL only renders convex polygons; those whose interior angles are each less than 180 degrees. A non-convex polygon is known as a concave polygon.
For concave polygons, GLU provides functions to translate them into a set of convex polygons.
We say that a polygon is planar if all its vertices lie in a single plane - for example, all triangles are planar.

4.3.1 Surface normal

The surface normal is a vector perpendicular to the plane of the polygon, we can use this to give a polygon a distinguishable front and back and also to describe its orientation in 3D space.
Orientation is an essential property used extensively in lighting calculations, collisions, culling, etc.

Finding the surface normal
Step 1: Choose a pair of sequential edges E_1 and E_2 and compute their vectors
Step 2: Invert the direction of the first edge so they now emanate from their shared vertex
Step 3: Their cross product gives the surface normal N = E_2 \times -E_1.
Step 4: Normalize N

4.3.2 Polygon soup

One way to represent a scene using polygons is to have a huge list of individual polygons, to color and draw in order, requiring individual manipulation. This is known as a polygon soup and has a number of undesirable properties, such as:

a. Waste of storage space as most models contain surfaces, not individual polygons, so we could share vertices rather than replicate them per polygon.
b. Loss of semantics: does a polygon belong to a row? or a table?
c. Leads to brute force rendering which makes interaction with the model complex

4.3.3 Meshes

Meshes are a linked group of polygons, which can be used to represent surfaces. They retain semantics, reduce storage by sharing vertices and edges, help with interaction and help with structuring the model so we can manipulate it more easily.

4.3.4 Triangle strips

Triangle strips are a collection of linked triangles, N linked triangles can be defined using N + 2 vertices - compared with 3N vertices if each triangle was defined separately

4.3.5 Triangle fans

Triangle fans are a collection of linked triangle that share a same vertex, which again be defined using N + 2 vertices instead of 3N

4.3.6 Quadrilateral strips

Quadrilateral strips are a collection of links quadrilaterals, N quadrilaterals can be defined using 2N + 2 vertices, compared with 4N separate vertices
Quadrilateral meshes are a collection of linked quadrilaterals, which is commonly used in terrain modelling and for approximating curved surfaces.
N,M quadrilaterals can be defined using (N + 1) \times (M + 1) vertices, compared with 4MN separate vertices

NB Quadrilaterals must be tessellated into triangles during rendering

4.4 Scan conversion

Scan conversion is the process of converting polygon edges in world coordinates into lines to display on a screen.

4.4.1 Scan-converting a line

To scan-convert a line, we sample the true geometry of the line, and approximate it using the nearest pixels available. Brenham (1965) developed a fast algorithm using only integer arithmetic, which is still in use today.

4.4.2 Scan-converting a polygon

When scan-converting a polygon, we know the polygon has been transformed by the viewing pipeline, so we know its (x, y, z) vertex coordinates in screen space, where the (x, y) coordinates correspond to a pixel position and the z coordinate is a measure of the vertex's distance from the eye.

4.4.3 Sweep-line algorithm

This algorithm steps down a pair of edges, starting in this example at (x_1, y_1), then goes down scanline by scanline, finding the start and end of the part of the scanline inside the triangle. This algorithm is efficient because we only need to compute the gradients of the edges px and qx once, at the beginning. However, it is a floating point algorithm, so we do need keep rounding to the pixel grid.

4.4.4 Hidden surface removal

When modelling a 3D world from a particular viewpoint, there will be some parts of the world we can see, and some parts we cannot see because they may be behind other surfaces. There are two main approaches to solving this problem:

- ✓ We can solve it in world space, i.e. we can try to work it out geometrically in 3D before drawing the result, this was the earlier approaches that was used and it is extremely hard.
- ✓ We can do it in display space, so, during scan-conversion, whenever we generate a pixel P, we determine whether some other 3D object, nearer to the eye, also maps to P. This is now the standard approach.

The z-buffer (depth-buffer)
The z-buffer keeps a record of the z-value of each pixel.
Initialise each pixel to the desired background colour
Initialise each z-buffer entry to MAXDEPTH
for each pixel P generated during scan-conversion of an object do
  if z-coordinate of P < ZBUFFER[P] then
    compute colour of P
    store colour in P
    store z-coordinate of P in ZBUFFER[P]
  else
    do nothing (since something else has already been mapped to P and is closer)

NB A lack of precision in the z-buffer leads to incorrect rendering of pixels with similar or identical z-values - this is known as z-fighting or stitching

4.5 Rendering Curves in OpenGL

OpenGL supports the drawing of curved surfaces through the use of evaluators. Evaluators can be used to construct curves and surfaces based on the Bernstein basis polynomials. In order to draw curves and surfaces using other basis polynomials (e.g., Hermite polynomials) the user program must transform that basis to a Bernstein basis. Example consider first one-dimensional evaluators.
The following steps are performed:

1. Define a one-dimensional evaluator with ***glMap1*()***
2. Enable it with glEnable()
   (Both functions are usually called as part of initialization.)

The function is evaluated at a series of points using ***glEvalCoord1()*** between a ***glBegin()*** and ***glEnd()*** block in the ***display()*** function. [This is similar to using ***glVertex*()***] ***glEvalCoord1()*** is usually called within a for loop. The function ***glMap1()*** defines a one-dimensional evaluator that evaluates the Bernstein polynomial of order n + 1, where n is the degree of the polynomial. ***void glMap1fd(GLenum target, TYPE t1, TYPE t2, Glint stride, GLint order, const TYPE *points)***; **t1** and **t2** specify the range for the variable t, **stride** specifies the number of entries between the beginning of one control point and the beginning of the next one in the data structure referenced by points. This allows control points to be embedded in arbitrary data structures. **Order** is the order of the polynomial, which is the same as the number of control points. ***points** is a pointer to the list of control points.

Parameter	Meaning
GL_MAP1_VERTEX_3	x, y, z vertex coordinates
GL_MAP1_VERTEX_4	x, y, z, w vertex coordinates
GL_MAP1_INDEX	color index
GL_MAP1_COLOR_4	R, G, B, A
GL_MAP1_NORMAL	normal coordinates
GL_MAP1_TEXTURE_COORD_1	s texture coordinates
GL_MAP1_TEXTURE_COORD_2	s, t texture coordinates
GL_MAP1_TEXTURE_COORD_3	s, t, r texture coordinates
GL_MAP1_TEXTURE_COORD_4	s, t, r, q texture coordinates

void glEvalCoord1{fd}(TYPE t); Evaluates the enabled one-dimensional function. The argument ***t*** is the value of the parameter.

The following code is from the program **bezcurve. c**:

```
#define STEPS 5

GLfloat ctrlpts[4][3] = {
        { -4.0, -4.0, 0.0}, { -2.0, 4.0, 0.0},
        {2.0, -4.0, 0.0}, {4.0, 4.0, 0.0}};

void init(void)
{
   glClearColor(0.0, 0.0, 0.0, 0.0);
   glShadeModel(GL_FLAT);
   glMap1f(GL_MAP1_VERTEX_3, 0.0, 1.0,
           3, 4, &ctrlpts[0][0]);
   glEnable(GL_MAP1_VERTEX_3);
}
```

Note: the **stride** parameter is **3**, while **order** is set to **4** (the **degree** is **3**).

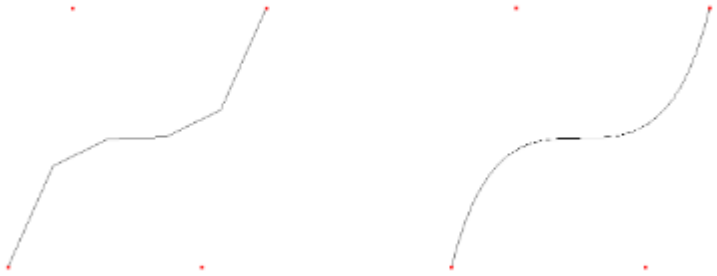
To display the following code is used:

```
void display(void)
{
   int i;

   glClear(GL_COLOR_BUFFER_BIT);
   glColor3f(1.0, 1.0, 1.0);
   glBegin(GL_LINE_STRIP);
      for (i = 0; i <= STEPS; i++)
         glEvalCoord1f((GLfloat) i / (GLfloat) STEPS);
   glEnd();

   /* Display the control points as dots. */
   glPointSize(5.0);
   glColor3f(0.0, 0.0, 1.0);
   glBegin(GL_POINTS);
      for (i = 0; i < 4; i++)
         glVertex3fv(&ctrlpts[i][0]);
   glEnd();
   glFlush();
}
```

The results of executing the code is as follows:



On the left, the constant **STEPS** is set to **5**; on the right it is set to **30**.

It is common for coordinate values to be evenly spaced, as they were in the previous example. In this case, the functions ***glMapGrid1()*** and ***glEvalMesh1()*** are useful.

void glMapGrid1{fd}(GLint n, TYPE t1, TYPE t2); Defines a grid that goes from **t1** to **t2** in **n** evenly spaced steps.

void glEvalMesh1(GLenum mode, GLint p1, GLint p2); Applies the currently defined grid to all enabled evaluators. The mode can be either **GL_POINT** or **GL_LINE**, depending on whether points or a connected line is required. The call has exactly the same effect as issuing a **glEvalCoord1()** for each of the steps from **p1** to **p2**, inclusive.

The previous code could be rewritten as:

```
void init(void)
{
   glClearColor(1.0, 1.0, 1.0, 0.0);
   glShadeModel(GL_FLAT);
   glMap1f(GL_MAP1_VERTEX_3, 0.0, 1.0, 3, 4, &ctrlpts[0][0]
   glEnable(GL_MAP1_VERTEX_3);
   glMapGrid1f(STEPS, 0.0, 1.0);
}
void display(void)
{
   glClear(GL_COLOR_BUFFER_BIT);
   glColor3f(1.0, 0.0, 0.0);
   glEvalMesh1(GL_LINE, 0, STEPS);

   /* Display control points as dots. */
         .
         .
         .
   glFlush();
}
```

4.6 Two dimensional evaluators

Two-dimensional evaluators evaluate the Bernstein polynomials for two parameters, u and v. These evaluators are used as follows:

1. Define the evaluator(s) with glMap2*()
2. Enable them by passing the appropriate value to glEnable()
3. Invoke them either by calling glEvalCoord2() between a glBegin and glEnd pair, or by specifying and applying a mesh with glMapGrid2() and glEvalMesh2().

void glMap2{fd}(GLenum target, TYPE u1, TYPE u2, GLintustride, GLint uorder, TYPE v1, TYPE v2, GLint vstride,GLint vorder, TYPE *points) The target parameter is as shown in the earlier table, exceptMAP1is replaced by MAP2.u1,u2,v1, and v2are the ranges for u and v. ustride and vstride are the "distance" to the next u and v values in the array of control points. Uorder and vorder are the order parameters, which also give the number of control points: uorder × vorder. For example, you could have control points for a number of different shapes stored in a single array GLfloat ctrlpts[100][100][3];To select the 4×4 subset (assume uorder=vorder= 4)starting at[20][10], you would specify that...ustride should be 3*100 and vstride should be 3 (C stores arrays in row major order).*points should be set to & ctrlpts[20][10][0].

void glEvalCoord2{fd}(TYPE u, TYPE v);Causes evaluation of the enabled two-dimensional maps. The arguments u and v are values for the domain coordinates. If either of the vertex evaluators is enabled (GL_MAP2_VERTEX_3orGL_MAP2_VERTEX_4), then the normal to the surface is computed analytically. This normal is associated with the generated vertex if automatic normal generation has been enabled using glEnable(GL_AUTO_NORMAL). If it is disabled, the corresponding enabled normal map is used. If no such map exists, the current normal is used.

The following code is from the example bezsurf2.c. Nine curved lines are drawn in each direction, with each one composed of 30 segments. The red lines vary in u, the blue ones vary inv. (The parameter maxJ is controlled by the user to illustrate the drawing order).

```
GLfloat ctrlpoints[4][4][3] = {
   {{-1.5, -1.5, 4.0}, {-0.5, -1.5, 2.0},
    {0.5, -1.5, -1.0}, {1.5, -1.5, 2.0}},
   {{-1.5, -0.5, 1.0}, {-0.5, -0.5, 3.0},
    {0.5, -0.5, 0.0}, {1.5, -0.5, -1.0}},
   {{-1.5, 0.5, 4.0}, {-0.5, 0.5, 0.0},
    {0.5, 0.5, 3.0}, {1.5, 0.5, 4.0}},
   {{-1.5, 1.5, -2.0}, {-0.5, 1.5, -2.0},
    {0.5, 1.5, 0.0}, {1.5, 1.5, -1.0}}
};
```

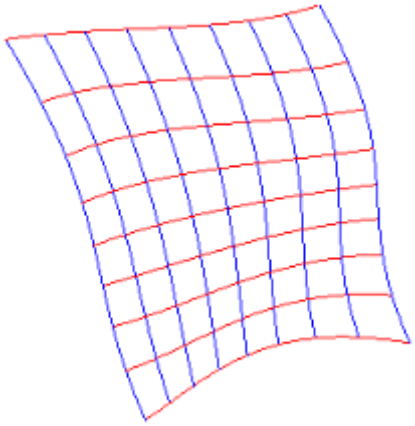
```
void display(void)
{
   int i, j;

   glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
   glPushMatrix ();
   glRotatef(15.0, 1.0, 1.0, 1.0);
   for (j = 0; j <= maxJ; j++) {
      glColor3f(1.0, 0.0, 0.0);
      glBegin(GL_LINE_STRIP);
      for (i = 0; i <= 30; i++)
         glEvalCoord2f((GLfloat)i/30.0, (GLfloat)j/8.0);
      glEnd();

      glColor3f(0.0, 0.0, 1.0);
      glBegin(GL_LINE_STRIP);
      for (i = 0; i <= 30; i++)
         glEvalCoord2f((GLfloat)j/8.0, (GLfloat)i/30.0);
      glEnd();
   }
   glPopMatrix ();
   glFlush();
}


void init(void)
{
   glClearColor (0.0, 0.0, 0.0, 0.0);
   glMap2f(GL_MAP2_VERTEX_3, 0, 1, 3, 4,
           0, 1, 12, 4, &ctrlpoints[0][0][0]);
   glEnable(GL_MAP2_VERTEX_3);
   glEnable(GL_DEPTH_TEST);
   glShadeModel(GL_FLAT);
}
```

The output of the above code is as shown below:



Lesson 4 Discussion Session

Use WebGL to create a program that has two dimensional curve output like the one shown below:

